The Narrative Structure of the *Popol Wuj*:A Statistical Approach

Rafael C. Alvarado | Spring 2025

Preliminaries

Import code libraries

```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import plotly.express as px
import seaborn as sns

from sklearn.feature_extraction.text import CountVectorizer, TfidfTransformer
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.decomposition import NMF, PCA, LatentDirichletAllocation as LDA
from sklearn.cluster import KMeans

from great_tables import GT
```

```
import sys; sys.path.append("../../local_lib/")

import hac
from hac import HAC
from heatmap import plot_grid, plot_map, CorrelationHeatMap as CHM
```

Set some hyperparameters

```
px_template = 'plotly_white'

# Number of equal size chunks to divide the TOKEN table
n_chunks = 60
# n_chunks = 100

# Use if parsing strings of Mayan
# token_pattern = r"(?u)\b\w[\w']*\b"

# These are used with CounterVectorizer
min_ngram = 1
max_ngram = 1

# The limit below which words will be excluded from models
dh_thresh_agg = 'mean' # Use 'median' for much less severe cut
```

Get and prepare the data

Read in CHAP and TOKEN

```
CHAP = pd.read_csv("christenson-CHAP-with-text.csv").set_index("chap_num")
TOKEN = pd.read_csv("christenson-TOKEN_QUC.csv").set_index(['chap_num','line_num','token_num
```

```
# TOKEN[TOKEN.term_str.str.contains("'$", regex=True)]
```

Create CHUNK

```
TOKEN['chunk_id'] = (
    pd.DataFrame(np.array_split(TOKEN.index, n_chunks))
    .stack()
    .reset_index()
    .level_0
    .values + 1
)
```

```
TOKEN.head(10)
```

chap_num	line_num
1	1
	2
	3

```
CHUNK = (
    TOKEN
    .groupby('chunk_id')
    .term_str
    .count()
    .to_frame('n_tokens')
)
```

```
CHUNK.value_counts('n_tokens')
```

```
n_tokens
489    41
488    19
Name: count, dtype: int64
```

```
# CHUNK
```

Create CHUNK CHAP Matrix

Chunk to Chapters Index

```
CHUNK_CHAP_MATRIX = TOKEN.reset_index().value_counts(['chap_num', 'chunk_id']).unstack(fill_
```

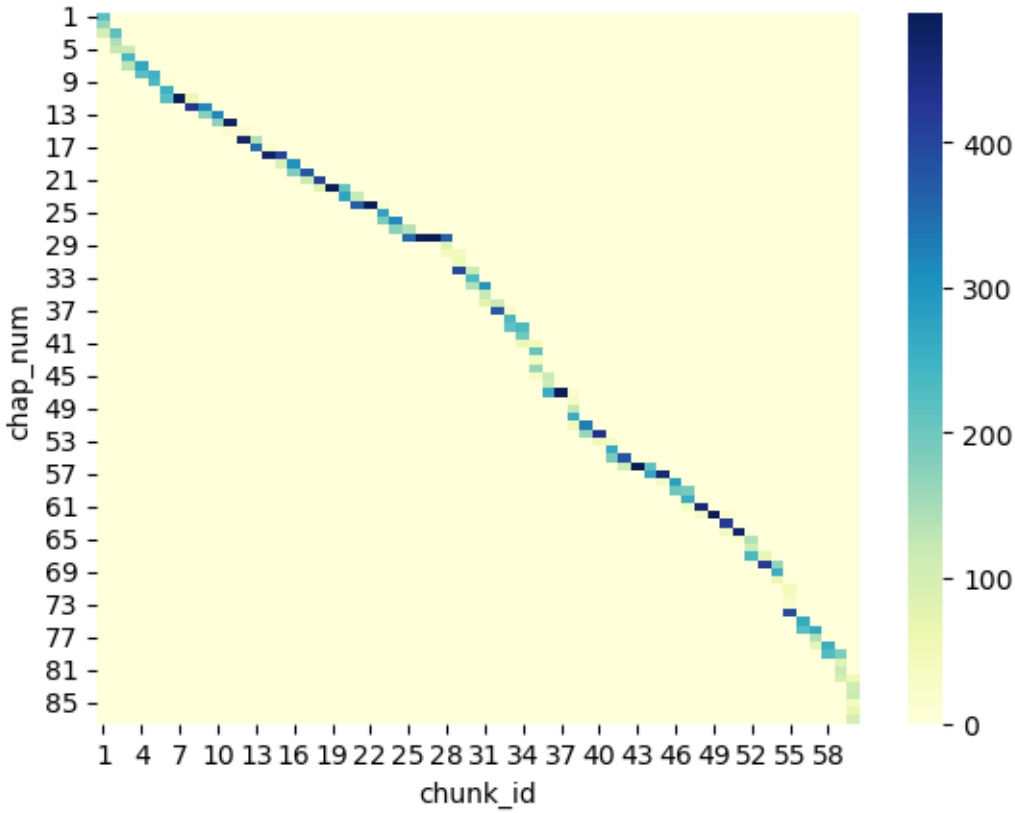
```
CHUNK_CHAP_MATRIX.sum(1).mean()
```

```
np.float64(337.02298850574715)
```

```
CHUNK_CHAP_MATRIX.sum().value_counts()
```

```
489.0    41
488.0    19
Name: count, dtype: int64
```

```
sns.heatmap(CHUNK_CHAP_MATRIX.fillna(0), cmap='YlGnBu');
```



Map CHAP to CHUNK

```
CHUNK_TO_CHAP = (
    CHUNK_CHAP_MATRIX
    .apply(lambda x: x.dropna().index.tolist())
    .apply(pd.Series)
    .stack()
    .to_frame('chap_num')
    .join(CHAP.chap_title, on='chap_num')
)
CHUNK_TO_CHAP.index.names = ['chunk_id', 'chap_ord']
CHUNK_TO_CHAP['chap_label'] = CHUNK_TO_CHAP.chap_num.astype(int).astype(str) + ': ' + CHUNK_T
CHUNK_TO_CHAP.chap_num = CHUNK_TO_CHAP.chap_num.astype(int)
```

```
CHUNK_TO_CHAP
```

chunk_id	chap_ord	cha
	0	1
1	1	2
	2	3
2	0	3
	1	4
...	...	...
	1	83
	2	84
60	3	85
	4	86
	5	87

Chapter Labels for Chunks

```
CHUNK['short_label'] = CHUNK_TO_CHAP.groupby('chunk_id').chap_num.apply(lambda x: '[' + str(x
CHUNK['long_label'] = CHUNK_TO_CHAP.groupby('chunk_id').chap_label.apply(lambda x: '[' + str
```

```
CHUNK.head()
```

chunk_id	n_tokens	short_label	long_label
1	489	[1] 1 2 3	[1] 1: Preamble 2: The Primordial World 3: The...
2	489	[2] 3 4 5	[2] 3: The Creation of the Earth 4: The Creati...
3	489	[3] 5 6 7	[3] 5: The Fall of the Animals 6: The Creation...
4	489	[4] 7 8	[4] 7: The Creation of the Effigies of Carved ...
5	489	[5] 8 9	[5] 8: The Fall of the Effigies of Carved Wood...

Map CHUNK to CHAP

```
CHAP['chunk_ids'] = CHUNK_CHAP_MATRIX.stack().reset_index().groupby('chap_num').chunk_id.appl
```

```
CHAP.head()
```

chap_num	chap_title	page_num	xom_start	xom_end	xom_start_fixed	xom_en
1	Preamble	59	1	96	1	96
2	The Primordial World	67	97	154	97	154
3	The Creation of the Earth	70	155	274	155	274
4	The Creation of the Animals	74	275	339	275	339
5	The Fall of the Animals	76	340	433	340	433

Craate vector spaces

Create BOW for CHAP and CHUNK

```
BOW_CHAP = TOKEN.groupby(['chap_num', 'term_str']).term_str.count()
BOW_CHUNK = TOKEN.groupby(['chunk_id', 'term_str']).term_str.count()
```

Create CTM

```
CTM = BOW_CHUNK.unstack(fill_value=0)
```

```
CTM.sample(10).T.sample(10).T.style.background_gradient(axis=None, cmap='Blues')
```

Table 5

term_str chunk_id	k'oj	muqikil	t'isb'al	xesaqir	cham	taqchi'b'al	kakib'an	kitze	chujujunal	chiqatzaq
24	0	0	0	0	0	0	0	0	0	0
20	0	0	0	0	0	0	0	0	0	0
60	0	0	0	0	0	0	0	0	0	0
32	0	0	0	0	0	0	0	0	0	0
41	0	0	0	0	0	0	1	3	0	0
36	0	0	0	0	0	0	0	3	0	0
31	0	0	0	0	0	0	0	0	0	0
25	0	0	0	0	0	0	0	0	0	0
12	0	0	0	0	0	0	0	0	0	0
11	0	0	0	0	0	0	0	0	0	0

Vocabulary

Extract VOCAB from CTM

```
VOCAB = CTM.sum().to_frame('n')
VOCAB.index.name = 'term_str'
VOCAB['grams'] = VOCAB.apply(lambda x: len(x.name.split()), axis=1)
```

```
VOCAB
```

term_str	n	grams
a	23	1
a'	12	1
a'on	2	1
ab'aj	31	1
ab'ajil	1	1
...	...	...
yoq'otajinaq	1	1
yujuj	2	1

term_str	n	grams
yuq	1	1
yuyub'	2	1
zuywa	1	1

Compute DFIDF

```
VOCAB['df'] = CTM.astype(bool).sum()
VOCAB['dp'] = VOCAB.df / n_chunks
VOCAB['dh'] = VOCAB.dp * np.log2(1/VOCAB.dp)
```

```
px.scatter(VOCAB.reset_index(), 'n', 'dh',
           log_x=True,
           width=750, height=500,
           hover_name='term_str',
           marginal_y='histogram',
          title="Term Significance (<i>dh</i>) v Frequency (<i>n</i>)")
```

```
Unable to display output for mime type(s): application/vnd.plotly.v1+json
```

```
px.scatter(VOCAB.reset_index(), 'df', 'dh',
           log_x=False,
           width=750, height=500,
           hover_name='term_str',
           marginal_y='histogram',
          title="Term Significance (<i>dh</i>) v Doc Frequency (<i>df</i>)")
```

```
Unable to display output for mime type(s): application/vnd.plotly.v1+json
```

```
VOCAB.describe()
```

	n	grams	df	dp	dh
count	3767.000000	3767.0	3767.000000	3767.000000	3767.000000
mean	7.783647	1.0	3.197239	0.053287	0.156108
std	44.613965	0.0	6.828925	0.113815	0.106787

	n	grams	df	dp	dh
min	1.000000	1.0	1.000000	0.016667	0.000000
25%	1.000000	1.0	1.000000	0.016667	0.098448
50%	1.000000	1.0	1.000000	0.016667	0.098448
75%	3.000000	1.0	2.000000	0.033333	0.163563
max	1161.000000	1.0	60.000000	1.000000	0.530735

Define Significant Terms

```
max_entropy = VOCAB.dh.agg(dh_thresh_agg)
```

```
dh_thresh_agg, max_entropy
```

```
('mean', np.float64(0.15610788313935567))
```

```
VOCAB['sig'] = VOCAB.dh >= max_entropy
```

```
VOCAB[VOCAB.sig == True].sample(10, weights='dh')
```

term_str	n	grams	df	dp	dh	sig
saqirik	24	1	13	0.216667	0.478064	True
xkiya'o	3	1	3	0.050000	0.216096	True
in	40	1	15	0.250000	0.500000	True
xub'ano	5	1	2	0.033333	0.163563	True
lab'e	4	1	4	0.066667	0.260459	True
lolmet	5	1	3	0.050000	0.216096	True
xe'uchaxik	23	1	16	0.266667	0.508504	True
xk'is	17	1	10	0.166667	0.430827	True
kib'il	18	1	15	0.250000	0.500000	True
pujuy	2	1	2	0.033333	0.163563	True

```
'Number of significant terms', VOCAB[VOCAB.sig].shape[0]
```

```
('Number of significant terms', 1272)
```

```
'Number of rejected words', len(VOCAB[~VOCAB.sig])
```

```
('Number of rejected words', 2495)
```

```
'Percent VOCAB used', round((len(VOCAB[VOCAB.sig]) / len(VOCAB)), 2) * 100
```

```
('Percent VOCAB used', 34.0)
```

Provide gloss feature

```
VOCAB['gloss'] = None
```

Term Significance

Compute TFIDF over non-stopwords

We weight the significance of words in each chunk using TFIDF.

TFIDF is an established method for estimating the significance of a word in a context by weighing its local frequency in a context against its global frequency in the corpus.

- To the degree that a word is frequent in a specific context and the general corpus, it is not considered siginficant.
- To the degree that a word is frequent in a specific context and infrequenct in the general corpus, it is considered significant.

```
CTMX = CTM[VOCAB[VOCAB.sig].index]
```

```
tfidf_engine = TfidfTransformer(norm='l2', use_idf=True, smooth_idf=True)
TFIDF = pd.DataFrame(tfidf_engine.fit_transform(CTMX).toarray(), columns=CTMX.columns, index=
```

```
# TFIDF.sample(10).sort_index().T.sample(5).sort_index().T.style.background_gradient(axis=No
```

Hierarchical Clusters

Create TFIDF_SIM

Cluster CHUNK docs by TFIDF.

We generate a square matrix of documents by documents showing pairwise similarities.

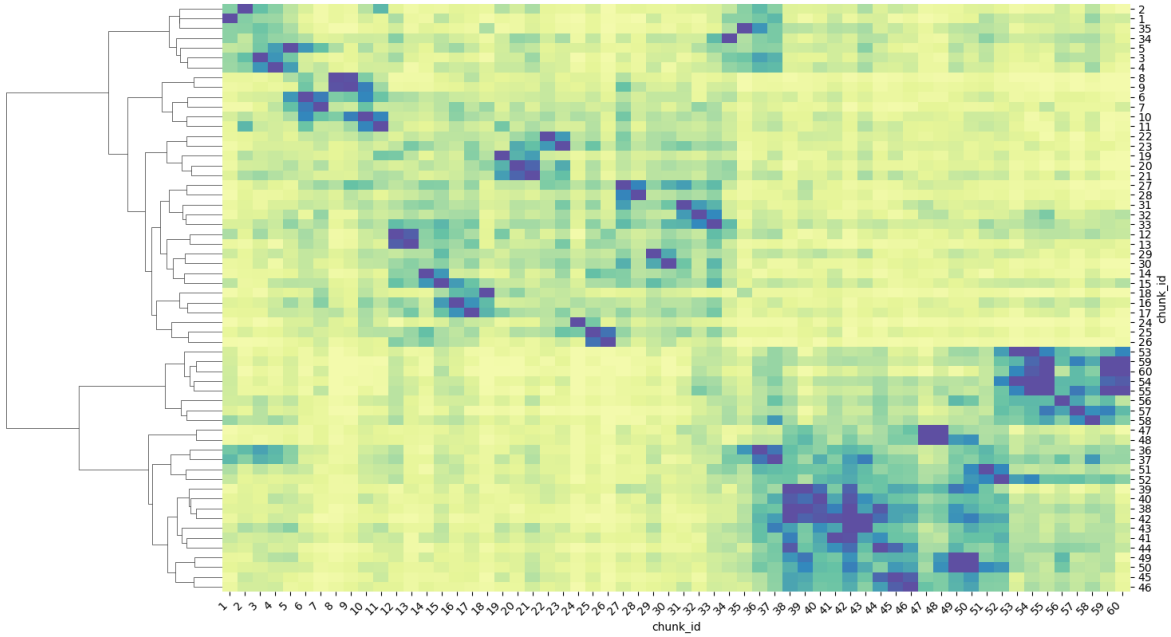
```
TFIDF_SIM = pd.DataFrame(cosine_similarity(TFIDF), index=TFIDF.index, columns=TFIDF.index)
```

View heatmap of document clusters

A heatmap of the same space provides some insight into the nature of the clusters.

```
# sns.clustermap?
```

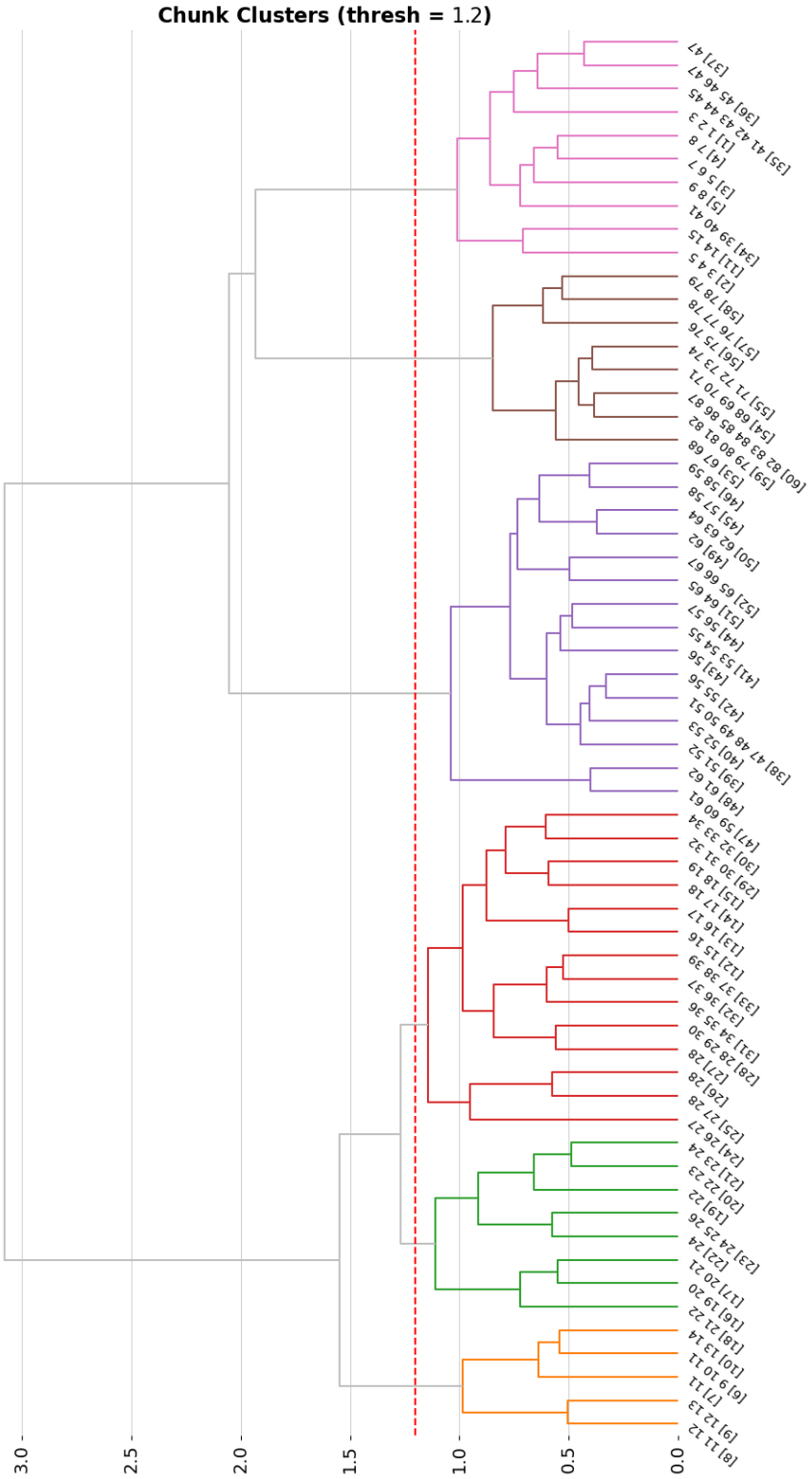
```
fig = sns.clustermap(TFIDF_SIM,
               method='ward',
               metric='euclidean',
               cmap='Spectral',
               center=0,
               cbar_pos=None,
               col_cluster=False,
               robust=True,
               # z_score=1,
               xticklabels=True,
               yticklabels=True,
                    figsize=(15,10))
plt.setp(fig.ax_heatmap.get_xticklabels(), rotation=0, ha="right")
plt.xticks(rotation=45)
fig.ax_col_dendrogram.set_visible(False)
plt.show()
```



Generate HAC

```
tfidf_hac = HAC(TFIDF_SIM, labels=CHUNK['short_label'].to_list())
tfidf_hac.get_sims()
tfidf_hac.get_tree()
tfidf_hac.color_thresh = 1.2
tfidf_hac.get_cluster_labels()
```

```
tfidf_hac.plot_tree()
plt.title(f"Chunk Clusters (thresh = {tfidf_hac.color_thresh}", y=1.01)
# ax = plt.gca()
plt.xticks(rotation=90)
plt.yticks(rotation=140, fontsize=10)
plt.title("Chunk Clusters (thresh = $1.2$)", fontsize=16, fontweight='heavy')
plt.show()
```


Chunk Clusters (thresh = 1.2)
3.0
2.5
2.0
1.5
1.0
0.5
0.0
[8] 11 12
[9] 12 13
[7] 11
[6] 9 10 11
[10] 13 14
[18] 21 22
[16] 19 20
[17] 20 21
[22] 24
[23] 24 25 26
[19] 22
[20] 22 23
[21] 23 24
[24] 26 27
[25] 27 28
[26] 28
[27] 28
[28] 28 29 30
[31] 34 35 36
[32] 36 37
[33] 37 38 39
[12] 15 16
[13] 16 17
[14] 17 18
[15] 18 19
[29] 30 31 32
[30] 32 33 34
[47] 59 60 61
[48] 61 62
[39] 51 52
[40] 52 53
[38] 47 48 49 50 51
[42] 55 56
[43] 56
[41] 53 54 55
[44] 56 57
[51] 64 65
[52] 65 66 67
[49] 62
[50] 62 63 64
[45] 57 58
[46] 58 59
[53] 67 68
[59] 79 80 81 82
[60] 82 83 84 85 86 87
[54] 68 69 70 71
[55] 71 72 73 74
[56] 75 76
[57] 76 77 78
[58] 78 79
[2] 3 4 5
[11] 14 15
[34] 39 40 41
[5] 8 9
[3] 5 6 7
[4] 7 8
[1] 1 2 3
[35] 41 42 43 44 45
[36] 45 46 47
[37] 47

Interpretation

- Two main groups.
- Visual inspection shows this corresponds to the first and second halves of the book consistent with how the text is divided by all editions.
- The exception is the group containing chs 1-8 and 39-46 – this group reflects the theme of creation, which begins both the first and second halves.
- This "echo" marks are thematic touch point.

Apply cluseter labels to chunks and chapters

```
CHUNK['cluster_label'] = tfidf_hac.CLUSTER_LABELS
TOKEN['cluster_label'] = TOKEN.chunk_id.map(CHUNK.cluster_label) # BETTER THAN JOIN
CHAP['cluster_label'] = (
    TOKEN
    .reset_index()
    .value_counts(['chap_num','cluster_label'])
    .unstack(fill_value=0)
    .idxmax(1)
)
```

```
# TOKEN.head()
```

Create CLUSTER table

```
CLUSTER = CHUNK.cluster_label.value_counts().to_frame('n_chunks')
```

```
CLUSTER
```

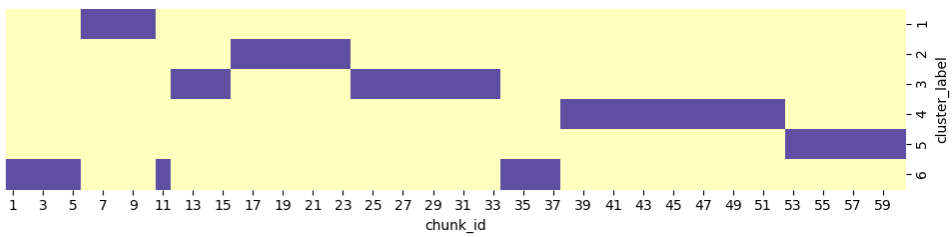
cluster_label	n_chunks
4	15
3	14
6	10
2	8
5	8
1	5

```
CLUSTER_CHUNK = TOKEN.groupby(['cluster_label','chunk_id']).chunk_id.count().unstack(fill_val
```

```
# CLUSTER_CHUNK
```

```
# px.imshow(CLUSTER_CHUNK, color_continuous_scale=px.colors.diverging.Spectral, color_contin
```

```
sns.clustermap(CLUSTER_CHUNK,
                    cmap='Spectral', method='ward',
                     cbar_pos=None, center=0,
                     row_cluster=False,
                     col_cluster=False,
                    figsize=(12,3))
plt.show()
```



Add order to CLUSTER

```
labels = {}
ord = 0
for lbl in CHUNK.sort_index().cluster_label.values:
    if lbl not in labels:
        ord += 1
        labels[lbl] = ord
```

```
CLUSTER['ord'] = pd.Series(labels)
```

```
CLUSTER.sort_values('ord')
```

cluster_label	n_chunks	ord
6	10	1
1	5	2
3	14	3
2	8	4
4	15	5
5	8	6

Create CLUSTER_TFIDF model

Cluster label glosses

We find significant words for each cluster based on mean TFIDF grouping by cluster.

This also creates a model from the labels; that is, each cluster label is associated with a distribution over words.

```
label_col = "cluster_label"
CLUSTER_TFIDF = (
    TFIDF
    .join(CHUNK[label_col])
    .groupby(label_col)
    .mean()
)
```

```
CLUSTER_TFIDF.T.sample(5).sort_index().T.style.background_gradient(axis=None)
```

Table 11

cluster_label	aq	xe	xe'uxik	xkitarij	xuya'o
1	0.008451	0.000000	0.000000	0.017177	0.000000
2	0.012459	0.006966	0.020534	0.000000	0.000000
3	0.003777	0.000000	0.006053	0.000000	0.000000
4	0.000000	0.000000	0.002555	0.000000	0.008319
5	0.000000	0.000000	0.000000	0.000000	0.000000
6	0.008657	0.011480	0.023793	0.005255	0.005266

```
VOCAB['max_cluster'] = CLUSTER_TFIDF.idxmax()
```

```
CLUSTER['gloss'] = CLUSTER_TFIDF.idxmax(1)
CLUSTER['top_terms'] = CLUSTER_TFIDF.apply(lambda x: ', '.join(x.sort_values(ascending=False
```

```
GT(CLUSTER.reset_index().sort_values('ord'))
```

cluster_label	n_chunks	ord	gloss	top_terms
6	10	1	kaj	kaj, b'it, aj, winaq, ulew, tz'aq, tz'ikin
1	5	2	sipakna	sipakna, kaqix, k'ajolab', wuqub', jul, o', much'
3	14	3	xib'alb'a	xib'alb'a, kame, junajpu, wuqub', xcha', kik', b'a
2	8	4	q'apoj	q'apoj, kati't, ati't, che', chowen, ch'o, b'atz'
4	15	5	b'alam	b'alam, amaq', tojil, kitze, b'i', majukutaj, jaq'awitz
5	8	6	ajaw	ajaw, ajawab', ja, aj, pop, nim, tinamit

Assign glosses by hand

```
VOCAB[VOCAB.dh > VOCAB.dh.quantile(.9)].sort_values('dh', ascending=False).head(10)
```

term_str	n	grams	df	dp	dh	sig	gloss	max_cluster
tzatz	35	1	22	0.366667	0.530735	True	None	4.0
su	27	1	23	0.383333	0.530276	True	None	2.0
chikiwach	35	1	23	0.383333	0.530276	True	None	2.0
amaq'	125	1	23	0.383333	0.530276	True	None	4.0
at	59	1	21	0.350000	0.530101	True	None	6.0
waral	30	1	21	0.350000	0.530101	True	None	6.0
wa'e	29	1	21	0.350000	0.530101	True	None	5.0
xib'alb'a	103	1	21	0.350000	0.530101	True	None	3.0
k'ab'awil	66	1	21	0.350000	0.530101	True	None	4.0
xkib'ano	29	1	21	0.350000	0.530101	True	None	2.0

```
glosses = {
    "b'alam": "jaguar",
    "ajaw": "lord",
    "kaj": "sky",
```

```
    "xib'alb'a": "underworld (earth)",
    "q'apoj": "maiden, virgin",
    "sipakna": "Zipacna",
    "kati't": "grandmother",
    "kaqix": "macaw",
    "tojil": "Tojil",
    "kame": "death",
    "pop": "mat",
    "q'alel": "magistrate",
    "b'it": "shap(er)",
    "ch'o": "rat",
    "qati't": "grandmother",
    "le'": "generation",
    "amaq'": "nations",
    "k'ajolab'": "boys",
    "alom": "children",
    "chowen": "artisan"
}

for key, val in glosses.items():
    VOCAB.loc[key, 'gloss'] = val
```

```
VOCAB[~VOCAB.gloss.isna()]
```

term_str	n	grams	df	dp	dh	sig	gloss	max_cluster
ajaw	137	1	25	0.416667	0.526264	True	lord	5.0
alom	17	1	7	0.116667	0.361612	True	children	6.0
amaq'	125	1	23	0.383333	0.530276	True	nations	4.0
b'alam	193	1	27	0.450000	0.518401	True	jaguar	4.0
b'it	35	1	9	0.150000	0.410545	True	shap(er)	6.0
ch'o	14	1	2	0.033333	0.163563	True	rat	2.0
chowen	27	1	7	0.116667	0.361612	True	artisan	2.0
k'ajolab'	87	1	17	0.283333	0.515505	True	boys	1.0
kaj	91	1	26	0.433333	0.522795	True	sky	6.0
kame	72	1	15	0.250000	0.500000	True	death	3.0
kaqix	43	1	10	0.166667	0.430827	True	macaw	1.0
kati't	30	1	7	0.116667	0.361612	True	grandmother	2.0
le'	24	1	5	0.083333	0.298747	True	generation	5.0
pop	42	1	9	0.150000	0.410545	True	mat	5.0

term_str	n	grams	df	dp	dh	sig	gloss	max_cluster
q'alel	24	1	7	0.116667	0.361612	True	magistrate	5.0
q'apoj	29	1	5	0.083333	0.298747	True	maiden, virgin	2.0
qati't	22	1	8	0.133333	0.387585	True	grandmother	2.0
sipakna	32	1	6	0.100000	0.332193	True	Zipacna	1.0
tojil	84	1	16	0.266667	0.508504	True	Tojil	4.0
xib'alb'a	103	1	21	0.350000	0.530101	True	underworld (earth)	3.0

Apply CLUSTER_TFIDF model to TOKEN

Group by CHUNK

```
CHUNK['max_cluster'] = (
    TOKEN
    .join(CLUSTER_TFIDF.T, on='term_str')
    .dropna()
    .groupby('chunk_id')[CLUSTER.index]
    .mean()
    .idxmax(1)
)
CHUNK['max_cluster_gloss'] = CHUNK.max_cluster.map(CLUSTER.gloss)
```

```
# CHUNK[['long_label', 'cluster_label', 'max_cluster','max_cluster_gloss']].style.background_
```

No difference.

Group by CHAP

```
CHAP['max_cluster'] = (
    TOKEN
    .join(CLUSTER_TFIDF.T, on='term_str')
    .dropna()
    .groupby('chap_num')[CLUSTER.index]
    .mean()
    .idxmax(1)
)
CHAP['max_cluster_gloss'] = CHAP.max_cluster.map(CLUSTER.gloss)
```

```
# CHAP[['chap_title', 'cluster_label', 'max_cluster', 'max_cluster_gloss']].style.background_
```

Principal Components

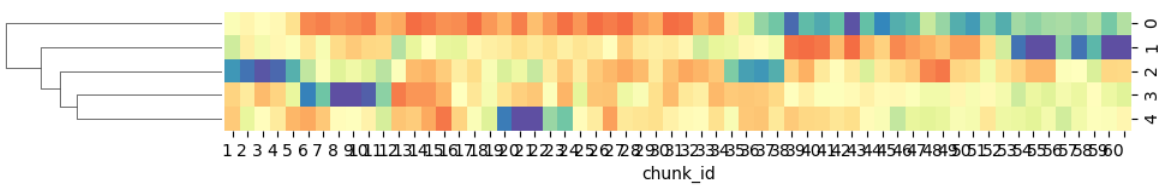
Apply PCA to TFIDF

PCA sheds light on the relationship between the clusters.

We add 1 to k if odd to ensure we have pairs to display.

```
n_comps = 5
pca_engine = PCA(n_components=5)
PCAX = pd.DataFrame(pca_engine.fit_transform(TFIDF), index=TFIDF.index)
PCAX.index.name = 'chunk_id'
LOADINGS = pd.DataFrame(pca_engine.components_.T * np.sqrt(pca_engine.explained_variance_), i
LOADINGS.index.name = 'term_str'
```

```
fig = sns.clustermap(PCAX.T,
               method='ward', metric='euclidean',
               cmap='Spectral', center=0, cbar_pos=None,
               col_cluster=False, robust=True,
               xticklabels=True, yticklabels=True,
                    figsize=(10,2))
plt.setp(fig.ax_heatmap.get_xticklabels(), rotation=0, ha="right")
fig.ax_col_dendrogram.set_visible(False)
```



Visualize compontents and clusters

```
X0 = CHUNK.join(PCAX)

def plot_pca(x, y):
```

```
    px.scatter(X0, x, y,
        text=X0.index,
        height=850, width=950,
        color=X0.max_cluster_gloss,
        # symbol=X0.max_topic,
        size = [1 for i in range(len(X0))],
        template=px_template,
        marginal_x='box',
        marginal_y='box').show()

    quantile = .99
    A = LOADINGS.loc[np.abs(LOADINGS[x]) >= np.abs(LOADINGS[x]).quantile(quantile), x]
    B = LOADINGS.loc[np.abs(LOADINGS[y]) >= np.abs(LOADINGS[y]).quantile(quantile), y]
    C = pd.concat([A,B], axis=1).index

    px.scatter(LOADINGS.loc[C].join(VOCAB), x, y,
        title=f"Quantile {quantile} Loadings for {x} and {y}",
        opacity=.5,
        text=C,
        size='n',
        template=px_template,
        height=850, width=950).show()
```

```
plot_pca(0,1)
```

```
Unable to display output for mime type(s): application/vnd.plotly.v1+json

Unable to display output for mime type(s): application/vnd.plotly.v1+json
```

```
plot_pca(2,3)
```

```
Unable to display output for mime type(s): application/vnd.plotly.v1+json

Unable to display output for mime type(s): application/vnd.plotly.v1+json
```

Component Histograms

```
def comp_box(comp_id):
    px.box(X0,
           x=comp_id,
           y='max_cluster_gloss',
           color='max_cluster_gloss',
           template=px_template,
           # height=400, width=600,
           title=f'PC {i}').show()
```

```
for i in range(n_comps):
    comp_box(i)
```

```
Unable to display output for mime type(s): application/vnd.plotly.v1+json

Unable to display output for mime type(s): application/vnd.plotly.v1+json

Unable to display output for mime type(s): application/vnd.plotly.v1+json

Unable to display output for mime type(s): application/vnd.plotly.v1+json

Unable to display output for mime type(s): application/vnd.plotly.v1+json
```

Topics

Generate NMF model

Topic modeling with NMF gives further insight into the content of the clusters and how they combine in documents, i.e. units of narrative.

As a form of soft clustering, it lends insight into how each text segment relates to each cluster.

It also corroborates the clustering.

Choose k based on cluster threshold

We choose k based on the chosen cut-off threshold for the clustering above.

```
k = len(set(tfidf_hac.CLUSTER_LABELS))
k
```

```
6
```

Create a list of columns for selection operations.

```
k_cols = [i for i in range(k)]
k_cols
```

```
[0, 1, 2, 3, 4, 5]
```

```
# NMF?
```

```
n_topic_terms = 10
n_topics = k
nmf_engine = NMF(n_components=n_topics,
                 max_iter=10000,
                 init='nndsvdar', # None, 'random', 'nndsvd', 'nndsvda', 'nndsvdar'
                 # alpha_W=.01,
                 # alpha_H=10,
                 solver='mu',
                 beta_loss='kullback-leibler'
                )

THETA = pd.DataFrame(nmf_engine.fit_transform(TFIDF), index=TFIDF.index)
THETA_SIM = pd.DataFrame(cosine_similarity(THETA), index=THETA.index, columns=THETA.index)

PHI = pd.DataFrame(nmf_engine.components_, columns=TFIDF.columns)
PHIX = PHI * np.log2(k/PHI.astype(bool).astype(int).sum())
PHIX = PHI

TOPIC = PHIX.T.apply(lambda x: ', '.join(x.sort_values(ascending=False).head(n_topic_terms).i
TOPIC.index.name = 'topic_id'
TOPIC['gloss'] = PHI.idxmax(1)
TOPIC['gloss2'] = PHIX.idxmax(1)
TOPIC['english'] = TOPIC.gloss.map(VOCAB.gloss)
TOPIC['english2'] = TOPIC.gloss2.map(VOCAB.gloss)
```

```
THETA_SEQ = THETA_SIM.unstack().to_frame('w')
THETA_SEQ.index.names = ['tmp', 'chunk_id']
THETA_SEQ = THETA_SEQ.query("chunk_id == tmp + 1")
THETA_SEQ = THETA_SEQ.reset_index().drop(columns=['tmp']).set_index('chunk_id')
THETA_SEQ['d'] = 1 - THETA_SEQ.w
```

```
# THETA_SEQ.d.plot.bar(figsize=(15,2), rot=0);
```

```
TOPIC
```

topic_id	top_terms	gloss	gloss2	english	english
0	b'alam, amaq', tojil, kitze, majukutaj, b'i', ...	b'alam	b'alam	jaguar	jaguar
1	xib'alb'a, kame, junajpu, kik', xcha', b'a, wu...	xib'alb'a	xib'alb'a	underworld (earth)	underw
2	ajaw, ja, ajawab', aj, pop, tinamit, nim, k'ic...	ajaw	ajaw	lord	lord
3	kaj, aj, b'it, winaq, ulew, tz'aq, alom, k'ajo...	kaj	kaj	sky	sky
4	kaqix, k'ajolab', sipakna, wuqub', jul, kab'ra...	kaqix	kaqix	macaw	macaw
5	kati't, qati't, ch'o, ati't, chowen, b'atz', c...	kati't	kati't	grandmother	grandm

```
# GT(TOPIC.sort_values('gloss2_ord')[['gloss2', 'english2', 'top_terms']])
```

```
CHUNK[f'top_topic_{k}'] = THETA.idxmax(1).values
```

Compare topics to clusters

```
CHUNK['max_topic'] = THETA.idxmax(1)
```

```
CHUNK[['max_cluster_gloss', 'max_topic']].value_counts().unstack(fill_value=0).style.backgrou
```

Table 16

max_cluster_gloss \ max_topic	0	1	2	3	4	5
ajaw	0	0	8	0	0	0
b'alam	14	0	1	0	0	0
kaj	0	0	0	8	1	0
q'apoj	0	2	0	0	0	6
sipakna	0	0	0	0	6	0
xib'alb'a	0	14	0	0	0	0

```
# THETA.join(CHUNK[['long_label', 'max_topic']]).style.background_gradient(cmap="YlGnBu", axi
```

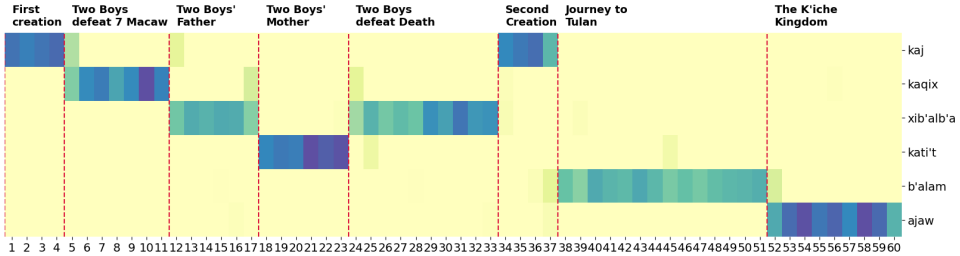
View topics over syuzhet: Thematic Structure

```
w = k / 2
TOPIC['gloss2_ord'] = [4, 2, 5, 0, 1, 3]
desired_order = list(TOPIC.sort_values('gloss2_ord').gloss2.values)
THETA.columns = TOPIC.gloss2
fig = sns.clustermap(THETA[desired_order].T,
                     cmap='Spectral', method='ward',
                     cbar_pos=None, center=0,
                     row_cluster=False,
                     col_cluster=False,
                    figsize=(20,w+2))
ax = plt.gca()
# plt.xlabel("Syntagm", fontsize=14, fontweight='heavy')
# plt.ylabel("Paradigm", fontsize=14, fontweight='heavy')
plt.xlabel("")
plt.ylabel("")
plt.xticks(fontsize=14)
plt.yticks(fontsize=14)

# Add chapter breaks
chapters = {
    (0,2) :  "First\ncreation",
    (4,3): "Two Boys\ndefeat 7 Macaw",
    (11,4): "Two Boys'\nFather",
    (17,3): "Two Boys'\nMother",
    (23,2): "Two Boys\ndefeat Death",
    (33,2): "Second\nCreation",
    (37,4): "Journey to\nTulan",
    (51,5): "The K'iche\nKingdom"
}

for x, y in chapters:
    ax.axvline(x, color='crimson', ls='--')
    ax.annotate(chapters[(x,y)], xy=(x,y), xytext=(x+.5, -.5), ha='left', fontsize=13, fontwe

plt.show()
```




```
chap_chunks = [ch[0] for ch in chapters]
CHUNK_TO_CHAP.loc[chap_chunks[1:]].reset_index()[['chunk_id', 'chap_num', 'chap_title']].set_
```

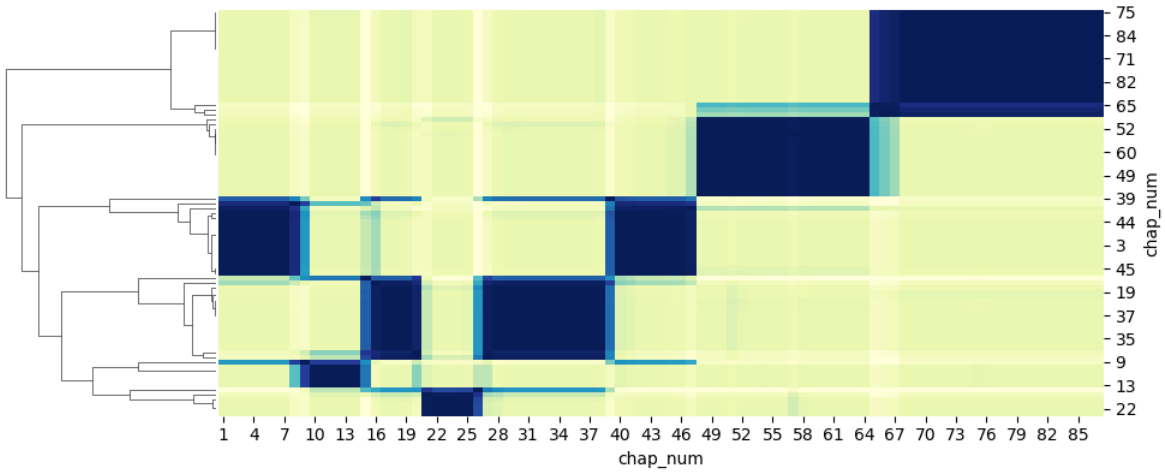
chunk_id	chap_num	ch
4	7	T
	8	T
11	14	T
	15	T
17	20	T
	21	La
23	24	H
	25	H
	26	T
	37	H
33	38	T
	39	T
37	47	T
51	64	T
	65	T

Classify chapters with NMF model

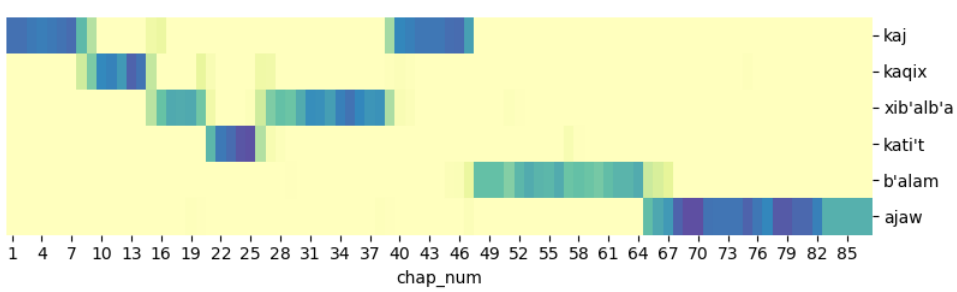
Model by THETA

```
CHAP_TOPIC_THETA = (
    TOKEN
    .join(THETA, on='chunk_id')[THETA.columns]
    .groupby(['chap_num'])
    .mean()
)
```

```
sns.clustermap(CHAP_TOPIC_THETA.T.corr(), cmap='YlGnBu', robust=True, col_cluster=False, row_
```



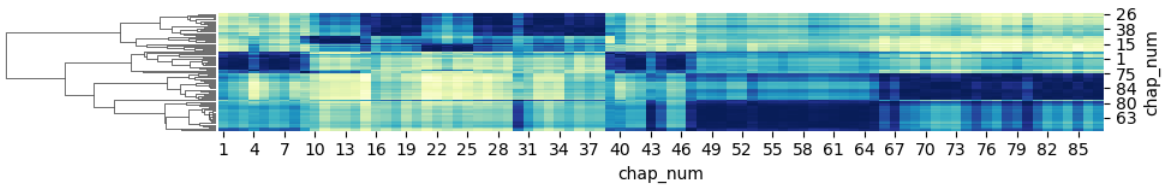

```
g = sns.clustermap(CHAP_TOPIC_THETA.T.loc[desired_order],
                    cmap='Spectral',
                    method='ward',
                    cbar_pos=None,
                    center=0,
                    row_cluster=False,
                    col_cluster=False,
                   figsize=(10,w))
plt.show()
```



Model by PHI

```
CHAP_TOPIC = (
    TOKEN
    .join(PHI.T, on='term_str')
    .dropna()
    .fillna(0)
    .groupby(['chap_num'])[k_cols]
    .mean()
)
```

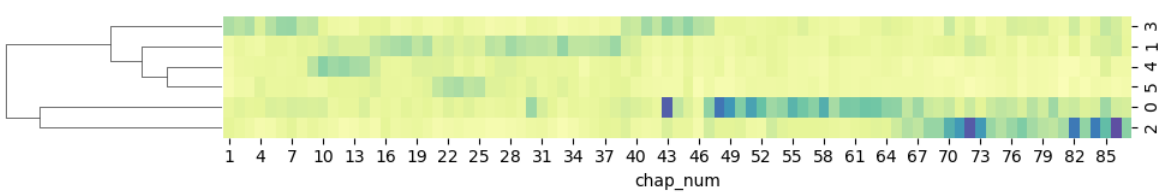
```
sns.clustermap(CHAP_TOPIC.T.corr(), cmap='YlGnBu', col_cluster=False, robust=False, cbar_pos=
```




```
from sklearn.preprocessing import normalize
```

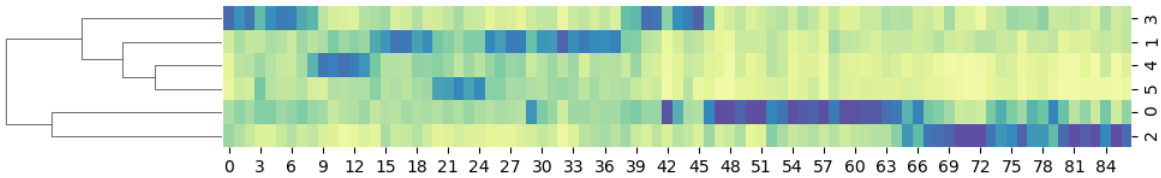
```
# CHAP_TOPIC.T
```

```
sns.clustermap(CHAP_TOPIC.T,
                      cmap='Spectral', method='ward',
                      cbar_pos=None, center=0,
                      row_cluster=True,
                      col_cluster=False,
                        robust=False,
                     figsize=(10,2));
```




```
sns.clustermap(normalize(CHAP_TOPIC.T, 'l2', axis=0),
                      cmap='Spectral', method='ward',
                      cbar_pos=None, center=0,
```

```
                    row_cluster=True,
                    col_cluster=False,
                      robust=True,
                  figsize=(10,2));
```




```
CHAP['max_topic'] = CHAP_TOPIC[k_cols].idxmax(1)
CHAP['max_topic_theta'] = CHAP_TOPIC_THETA.idxmax(1)
CHAP['max_topic_gloss'] = CHAP.max_topic.map(TOPIC.gloss)
```

```
CHAP[['max_cluster_gloss', 'max_topic_theta']].value_counts().unstack(fill_value=0).style.bac
```

Table 18

max_topic_theta max_cluster_gloss	ajaw	b'alam	kaj	kaqix	kati't	xib'alb'a
ajaw	21	0	0	0	0	0
b'alam	2	17	1	0	0	1
kaj	0	0	16	1	0	0
q'apoj	0	0	0	0	5	2
sipakna	0	0	0	5	0	0
xib'alb'a	0	0	0	0	1	15

Sequential correlations

Sequential Correlations of Chunks

We apply a simplified model of resonance.

```
THETA = THETA[desired_order]
```

```python
import plotly.graph_objects as go
from plotly.subplots import make_subplots
import numpy as np

# Compute color range for centered colormap
z = THETA.T.values
zmin, zmax = z.min(), z.max()
midpoint = 0
vmax = max(abs(zmin - midpoint), abs(zmax - midpoint))
zmin, zmax = midpoint - vmax, midpoint + vmax

# Create subplot layout
fig = make_subplots(
    rows=2, cols=1,
    shared_xaxes=True,
    row_heights=[0.6, 0.4],
    vertical_spacing=0.1,
    subplot_titles=("Gloss × Chunk Heatmap", "Chunk Distance from Previous")
)

# Heatmap
heatmap = go.Heatmap(
    z=z,
    x=THETA.T.columns,
    y=THETA.T.index,
    colorscale='Spectral',
    zmin=zmin,
    zmax=zmax,
    showscale=False
)
fig.add_trace(heatmap, row=1, col=1)

# Barplot of distances (by similarity)
bar = go.Bar(
    x=THETA_SEQ.index,  # assuming chunk_id is the index
    y=THETA_SEQ['d'],
    marker_color='royalblue'
)
fig.add_trace(bar, row=2, col=1)

# Layout adjustments
fig.update_layout(
```

```
    height=350,
    width=1000,
    margin=dict(t=60),
    showlegend=False
)

fig.update_annotations(font_size=14)
fig.update_yaxes(title_text="gloss", row=1, col=1)
fig.update_yaxes(title_text="d", row=2, col=1)
fig.update_xaxes(title_text="chunk_id", row=2, col=1)

fig.show()
```

```
Unable to display output for mime type(s): application/vnd.plotly.v1+json
```

Evidence for Br and Re break for Part IV – Midway through Ch. 57.

```
CHUNK_CHAP_MATRIX[[52]].dropna().join(CHAP.chap_title)
```

chap_num	52	chap_title
65	142.0	The Sons of the Progenitors Journey to Tulan
66	109.0	The Sons Receive Authority to Rule
67	237.0	The Migrations of the Quichés

```
CHUNK_CHAP_MATRIX[[38]].dropna().join(CHAP.chap_title)
```

chap_num	38	chap_title
47	18.0	The Creation of the Mothers of the Quiché Nation
48	33.0	The Arrival at Tulan
49	125.0	The Progenitors Receive their Gods
50	262.0	The Progenitors Leave Tulan
51	51.0	The Nations Ask for their Fire

```
CHUNK_CHAP_MATRIX[[34]].dropna().join(CHAP.chap_title)
```

chap_num	34	chap_title
39	236.0	The Miraculous Maize of Hunahpu and Xbalanque
40	197.0	The Apotheosis of the Sun, Moon, and Stars
41	56.0	The Creation of Humanity

Sequential Correlation among Chapters

```
CHAP_TOPIC_THETA_SIM = pd.DataFrame(cosine_similarity(CHAP_TOPIC_THETA), index=CHAP_TOPIC_THE
CHAP_TOPIC_THETA_SIM.index.names = ['tmp', 'chap_id']
CHAP_TOPIC_THETA_SIM = CHAP_TOPIC_THETA_SIM.query("chap_id == tmp + 1").reset_index().set_inc
CHAP_TOPIC_THETA_SIM['d'] = 1 - CHAP_TOPIC_THETA_SIM.w
# CHAP_TOPIC_THETA_SIM
```

```
CHAP_TOPIC_THETA
```

chap_num	b'alam	xib'alb'a	ajaw	kaj	kaqix	kati't
1	1.203897e-26	1.451182e-32	1.930933e-14	5.234777e-01	5.002757e-31	3.982824e-45
2	1.203897e-26	1.451182e-32	1.930933e-14	5.234777e-01	5.002757e-31	3.982824e-45
3	5.078138e-16	5.366087e-27	6.113613e-15	5.024277e-01	3.173519e-26	1.047986e-16
4	7.430853e-16	7.852203e-27	7.160420e-33	4.926752e-01	4.643795e-26	1.533521e-16
5	3.837244e-16	4.054843e-27	3.697594e-33	5.053264e-01	2.398025e-26	7.919001e-17
...	...	...	...	...	...	...
83	5.892949e-16	5.638698e-45	3.937618e-01	4.009543e-27	3.871056e-32	5.195948e-53
84	5.892949e-16	5.638698e-45	3.937618e-01	4.009543e-27	3.871056e-32	5.195948e-53
85	5.892949e-16	5.638698e-45	3.937618e-01	4.009543e-27	3.871056e-32	5.195948e-53
86	5.892949e-16	5.638698e-45	3.937618e-01	4.009543e-27	3.871056e-32	5.195948e-53
87	5.892949e-16	5.638698e-45	3.937618e-01	4.009543e-27	3.871056e-32	5.195948e-53

```
# Compute color range for centered colormap

# Create subplot layout
fig = make_subplots(
    rows=2, cols=1,
    shared_xaxes=True,
    row_heights=[0.6, 0.4],
    vertical_spacing=0.1,
```

```
    subplot_titles=("Glosses over Chpaters", "Chapter Distance from Previous")
)

# Heatmap
z = CHAP_TOPIC_THETA.T.values
zmin, zmax = z.min(), z.max()
midpoint = 0
vmax = max(abs(zmin - midpoint), abs(zmax - midpoint))
zmin, zmax = midpoint - vmax, midpoint + vmax
heatmap = go.Heatmap(
    z=z,
    x=CHAP_TOPIC.T.columns,
    y=TOPIC.gloss, #CHAP_TOPIC.T.index,
    colorscale='Spectral',
    zmin=zmin,
    zmax=zmax,
    showscale=False
)
fig.add_trace(heatmap, row=1, col=1)

# Barplot
X = CHAP_TOPIC_THETA_SIM.join(CHAP).reset_index()
bar = go.Bar(
    x = X.chap_id,
    y = X.d, #.rolling(2).mean(),
    marker_color='royalblue'
)
fig.add_trace(bar, row=2, col=1)

# Layout adjustments
fig.update_layout(
    height=330,
    width=1000,
    margin=dict(t=60),
    showlegend=False
)

fig.update_annotations(font_size=16)

# fig.update_yaxes(title_text="chapter", row=1, col=1)
fig.update_yaxes(title_text="topic", row=1, col=1)
fig.update_yaxes(title_text="d", row=2, col=1)
```

```
fig.update_xaxes(title_text="chapter number", row=2, col=1)

fig.show()
```

```
Unable to display output for mime type(s): application/vnd.plotly.v1+json
```

```
chap_nums = 8, 9, 10, 15, 16, 21, 26, 27, 39, 40, 48, 65
part_num = 1
for i, chap_num in enumerate(chap_nums):
    if i == 0 or chap_num > chap_nums[i-1] + 1:
        part_num += 1
        print('Part', part_num) #, '-' * 80)
    print("  ", str(chap_num).zfill(2), CHAP.loc[chap_num].chap_title)
```

```
Part 2
   08 The Fall of the Effigies of Carved Wood
   09 The Pride of Seven Macaw Before the Dawn
   10 The Fall of Seven Macaw and His Sons
Part 3
   15 The Tale of the Father of Hunahpu and Xbalanque
   16 The Deeds of One Hunahpu and Seven Hunahpu Above Xibalba
Part 4
   21 Lady Blood and the Miracle of the Maize
Part 5
   26 The Summons of Hunahpu and Xbalanque to Xibalba
   27 Hunahpu and Xbalanque Receive the Summons to Xibalba
Part 6
   39 The Miraculous Maize of Hunahpu and Xbalanque
   40 The Apotheosis of the Sun, Moon, and Stars
Part 7
   48 The Arrival at Tulan
Part 8
   65 The Sons of the Progenitors Journey to Tulan
```

```
CHAP['wdiff'] = CHAP_TOPIC_THETA_SIM.d
CHAP['new'] = (CHAP_TOPIC_THETA_SIM.d > .25).astype(bool)
CHAP.loc[1, 'new'] = True
```

Assign PART titles

```
PART = CHAP.loc[CHAP.new == True, ['chap_title']].reset_index()
PART.index.name = 'part_num'
PART = PART.reset_index()
PART.part_num = PART.part_num + 1
PART = PART.set_index('part_num')
PART = PART.sort_index()
```

```
PART
```

part_num	chap_num	chap_title
1	1	Preamble
2	15	The Tale of the Father of Hunahpu and Xbalanque
3	21	Lady Blood and the Miracle of the Maize
4	27	Hunahpu and Xbalanque Receive the Summons to X...
5	39	The Miraculous Maize of Hunahpu and Xbalanque
6	48	The Arrival at Tulan
7	62	The Nations Are Humiliated
8	65	The Sons of the Progenitors Journey to Tulan

```
CHAP['part_num'] = PART.reset_index().set_index('chap_num').part_num
CHAP.part_num = CHAP.part_num.ffill()
```

```
CHAP[['chap_title', 'chunk_ids', 'max_cluster', 'max_topic_theta', 'part_num', 'wdiff', 'new
```

Table 24

chap_num	chap_title	chunk_ids	max_cluster	m
1	Preamble	1	6	ka
2	The Primordial World	1	6	ka
3	The Creation of the Earth	1,2	6	ka
4	The Creation of the Animals	2	6	ka
5	The Fall of the Animals	2,3	6	ka
6	The Creation of the Mud Person	3	6	ka
7	The Creation of the Effigies of Carved Wood	3,4	6	ka

chap_num	chap_title	chunk_ids	max_cluster	m
8	The Fall of the Effigies of Carved Wood	4,5	6	ka
9	The Pride of Seven Macaw Before the Dawn	5,6	6	ka
10	The Fall of Seven Macaw and His Sons	6	1	ka
11	The Defeat of Seven Macaw	6,7,8	1	ka
12	The Deeds of Zipacna and the Four Hundred Boys	8,9	1	ka
13	The Defeat of Zipacna	9,10	1	ka
14	The Defeat of Cabracan	10,11	1	ka
15	The Tale of the Father of Hunahpu and Xbalanque	11,12	3	xi
16	The Deeds of One Hunahpu and Seven Hunahpu Above Xibalba	12,13	3	xi
17	The Summons of One Hunahpu and Seven Hunahpu to Xibalba	13,14	3	xi
18	The Descent of One Hunahpu and Seven Hunahpu into Xibalba	14,15	3	xi
19	The Maiden Lady Blood and the Tree of One Hunahpu	15,16	2	xi
20	The Ascent of Lady Blood from Xibalba	16,17	2	xi
21	Lady Blood and the Miracle of the Maize	17,18	2	ka
22	Hunahpu and Xbalanque in the House of the Grandmother	18,19,20	2	ka
23	The Fall of One Batz and One Chouen	20,21	2	ka
24	Hunahpu and Xbalanque in the Maizefield	21,22,23	2	ka
25	Hunahpu and Xbalanque Discover the Gaming Things	23	2	ka
26	The Summons of Hunahpu and Xbalanque to Xibalba	23,24	3	ka
27	Hunahpu and Xbalanque Receive the Summons to Xibalba	24,25	3	xi
28	The Descent of Hunahpu and Xbalanque into Xibalba	25,26,27,28	3	xi
29	Hunahpu and Xbalanque in the House of Cold	28	3	xi
30	Hunahpu and Xbalanque in the House of Jaguars	28,29	4	xi
31	Hunahpu and Xbalanque in the House of Fire	29	3	xi
32	Hunahpu and Xbalanque in the House of Bats	29,30	3	xi
33	The Head of Hunahpu is Restored	30	3	xi
34	The Deaths of Hunahpu and Xbalanque	30,31	3	xi
35	The Resurrection of Hunahpu and Xbalanque	31	3	xi
36	The Summons of Hunahpu and Xbalanque Before the Lords	31,32	3	xi
37	Hunahpu and Xbalanque Dance Before the Lords of Xibalba	32,33	3	xi
38	The Defeat of the Lords of Xibalba	33	3	xi
39	The Miraculous Maize of Hunahpu and Xbalanque	33,34	6	ka
40	The Apotheosis of the Sun, Moon, and Stars	34	6	ka
41	The Creation of Humanity	34,35	6	ka
42	The Discovery of Maize	35	6	ka
43	The First Four Men	35	4	ka
44	The Miraculous Vision of the First Men	35	6	ka
45	The Gratitude of the First Men	35,36	6	ka
46	The Displeasure of the Gods	36	6	ka
47	The Creation of the Mothers of the Quiché Nation	36,37,38	6	ka

chap_num	chap_title	chunk_ids	max_cluster	m
48	The Arrival at Tulan	38	4	b'
49	The Progenitors Receive their Gods	38	4	b'
50	The Progenitors Leave Tulan	38	4	b'
51	The Nations Ask for their Fire	38,39	4	b'
52	The Nations Are Deceived into Offering Themselves	39,40	4	b'
53	The People Gather on the Mountain Chi Pixab	40,41	4	b'
54	Their Suffering on the Mountain Chi Pixab	41	4	b'
55	The Progenitors Await the First Dawn	41,42	4	b'
56	The Appearance of the First Dawn	42,43,44	4	b'
57	Offerings Are Made to the Gods	44,45	4	b'
58	The Sacrificial Killings Begin	45,46	4	b'
59	The Nations Plot to Seduce the Gods	46,47	4	b'
60	The Seduction of Lady Lust and Lady Wailing	47	4	ka
61	The Nations Are Defeated by the Painted Robes	47,48	4	ka
62	The Nations Are Humiliated	48,49,50	4	b'
63	The Nations Are Defeated	50	4	b'
64	The Deaths of the Four Progenitors	50,51	4	b'
65	The Sons of the Progenitors Journey to Tulan	51,52	4	aj
66	The Sons Receive Authority to Rule	52	5	aj
67	The Migrations of the Quichés	52,53	4	aj
68	The Foundation of Chi Izmachi	53,54	5	aj
69	The Foundation of Cumarcah	54	5	aj
70	The Titles of the Cavec Lords	54	5	aj
71	The Titles of the Nihaib Lords	54,55	5	aj
72	The Titles of the Ahau Quiché Lords	55	5	aj
73	The Titles of the Zaquic Lords	55	5	aj
74	The Glory of the Lords of Cumarcah	55	5	aj
75	The Victories of Lord Quicab	56	5	aj
76	The Quichés Garrison the Conquered Regions	56,57	5	aj
77	The Titles Given to Those Who Were Ennobled	57	5	aj
78	The Names of the Gods' Houses	57,58	5	aj
79	The Prayer of the Lords to their Gods	58,59	5	aj
80	The Generations of Quiché Lords	59	5	aj
81	The Dynasty of Cavec Quiché Lords	59	5	aj
82	The Great Houses of the Cavec Lords	59,60	5	aj
83	The Dynasty of Nihaib Lords	60	5	aj
84	The Great Houses of the Nihaib Lords	60	5	aj
85	The Dynasty of Ahau Quiché Lords	60	5	aj
86	The Great Houses of the Ahau Quiché Lords	60	5	aj

87	The Three Great Stewards as Mothers of the Word	60	5	aj

CLUSTER

cluster_label	n_chunks	ord	gloss	top_terms
4	15	5	b'alam	b'alam, amaq', tojil, kitze, b'i', majukutaj, ...
3	14	3	xib'alb'a	xib'alb'a, kame, junajpu, wuqub', xcha', kik',...
6	10	1	kaj	kaj, b'it, aj, winaq, ulew, tz'aq, tz'ikin
2	8	4	q'apoj	q'apoj, kati't, ati't, che', chowen, ch'o, b'atz'
5	8	6	ajaw	ajaw, ajawab', ja, aj, pop, nim, tinamit
1	5	2	sipakna	sipakna, kaqix, k'ajolab', wuqub', jul, o', much'

TOPIC

topic_id	top_terms	gloss	gloss2	english	english
0	b'alam, amaq', tojil, kitze, majukutaj, k'ab'a...	b'alam	b'alam	jaguar	jaguar
1	xib'alb'a, kame, junajpu, kik', xcha', b'a, wu...	xib'alb'a	xib'alb'a	underworld (earth)	underw
2	ajaw, ja, ajawab', aj, pop, tinamit, nim, k'ic...	ajaw	ajaw	lord	lord
3	kaj, aj, b'it, winaq, ulew, tz'aq, alom, k'ajo...	kaj	kaj	sky	sky
4	kaqix, k'ajolab', sipakna, wuqub', jul, kab'ra...	kaqix	kaqix	macaw	macaw
5	kati't, qati't, ch'o, q'apojib', ati't, chowen...	kati't	kati't	grandmother	grandm

PART

part_num	chap_num	chap_title
1	1	Preamble
2	15	The Tale of the Father of Hunahpu and Xbalanque
3	21	Lady Blood and the Miracle of the Maize
4	27	Hunahpu and Xbalanque Receive the Summons to X...
5	39	The Miraculous Maize of Hunahpu and Xbalanque
6	48	The Arrival at Tulan
7	62	The Nations Are Humiliated
8	65	The Sons of the Progenitors Journey to Tulan

Conclusion

TOPICS

- The Parts
  - 7 Parts
  - Highlighting the role of Blood Woman
  - Optional Preamble and Coda
  - Each part is distinguished temporally and ontologically
- Parts in relation to chronotope
  - Before and after the dawn
  - Above and below the world
  - Transitions: ends adumbrate beginning in the manner of the Haab
- Thematic structure
  - Themes
    * K'aj: Creation
    * B'alam: Religion, Spiritual Authority
    * Ajaw: Rulership, Temporal Power
    * Xib'alb'a: The Underworld, Destruction, Two Boys and their parents
    * Kaqix: 7 Macaw
    * Katoi't: Blood Woman
  - Sequentially
    * Each major part has three parts: Creation, Part A, and Part B
    * The sequence matters: We go from the creation of the world to the creation of society
    * The sequence charts a lineage from the K'iche' kings to the creator gods.
  - Structurally
    * Two major parts corresponding to before and after the first dawn
    * See PCA
- Entropy
  - The goings on in Xibalba have the moist variability
- General

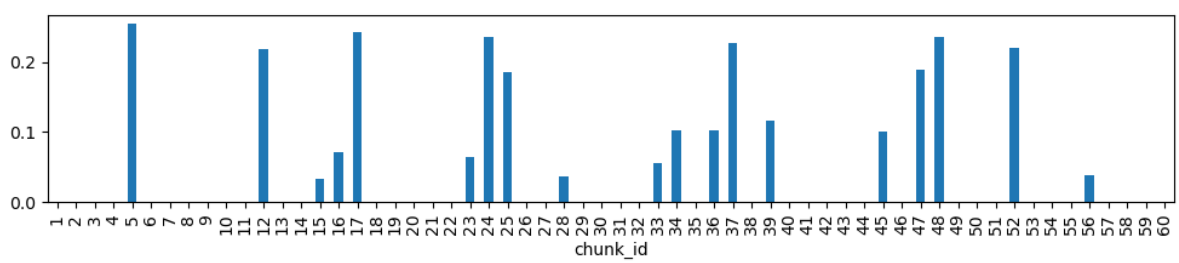
Save

```
savers = [
    (TFIDF, 'TFIDF'),
    (TFIDF_SIM, 'TFIDF_SIM'),
    (CHAP, 'CHAP_MOD'),
    (CHUNK, 'CHUNK_MOD'),
    (CHUNK_TO_CHAP, 'CHUNK_TO_CHAP'),
    (CHAP_TOPIC, 'CHAP_TOPIC')
]
```

```
for saver in savers:
    obj = saver[0]
    slug = saver[1]
    obj.to_csv(f"christenson-{slug}.csv", index=True)
```

Entropy

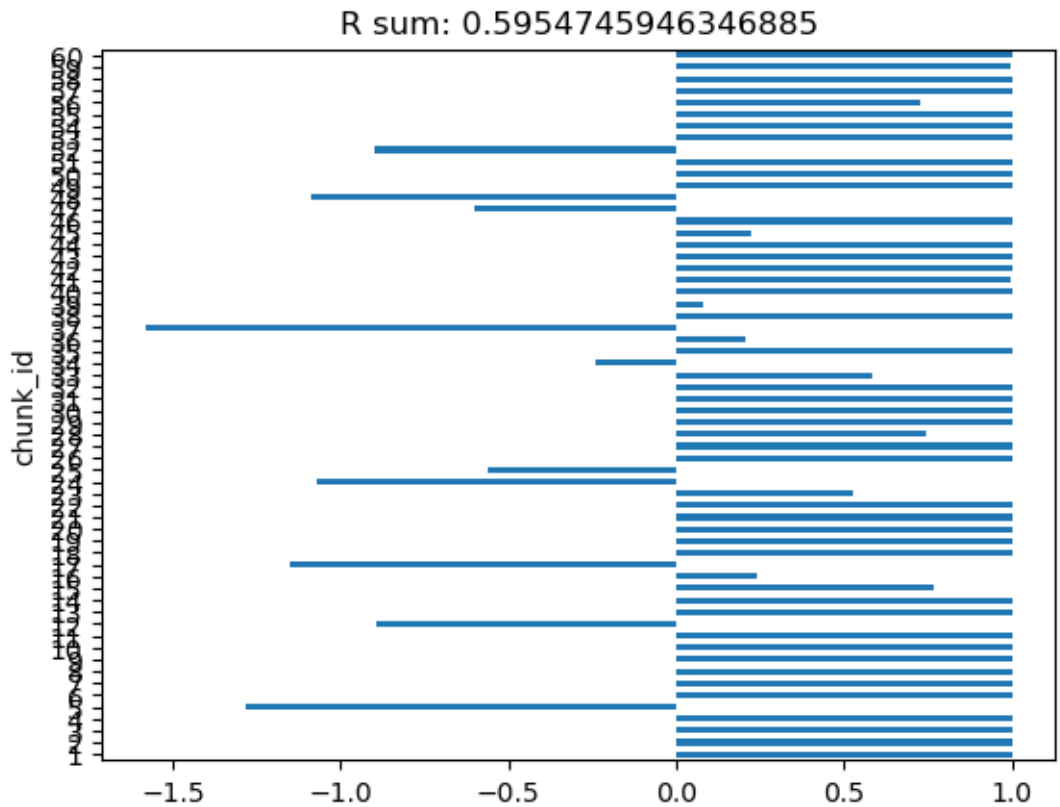
```
# M = CHAP_TOPIC.T
# M = CHAP_TOPIC_THETA.T
M = THETA.T
# M = PHI.T
P = (M / M.sum())
H = P * np.log2(1/P)
H.std().round(3).plot.bar(figsize=(12,2));
```




```
def compute_entropy(X):
    P = X / X.sum()
    H = (P * np.log2(1/P)).sum()
    Hmax =  1/len(X) * np.log2(len(X))
    R = 1 - H/Hmax
    R.plot.barh(title=f'R sum: {R.mean()}')
    # return R.mean()
```

```
# R
```

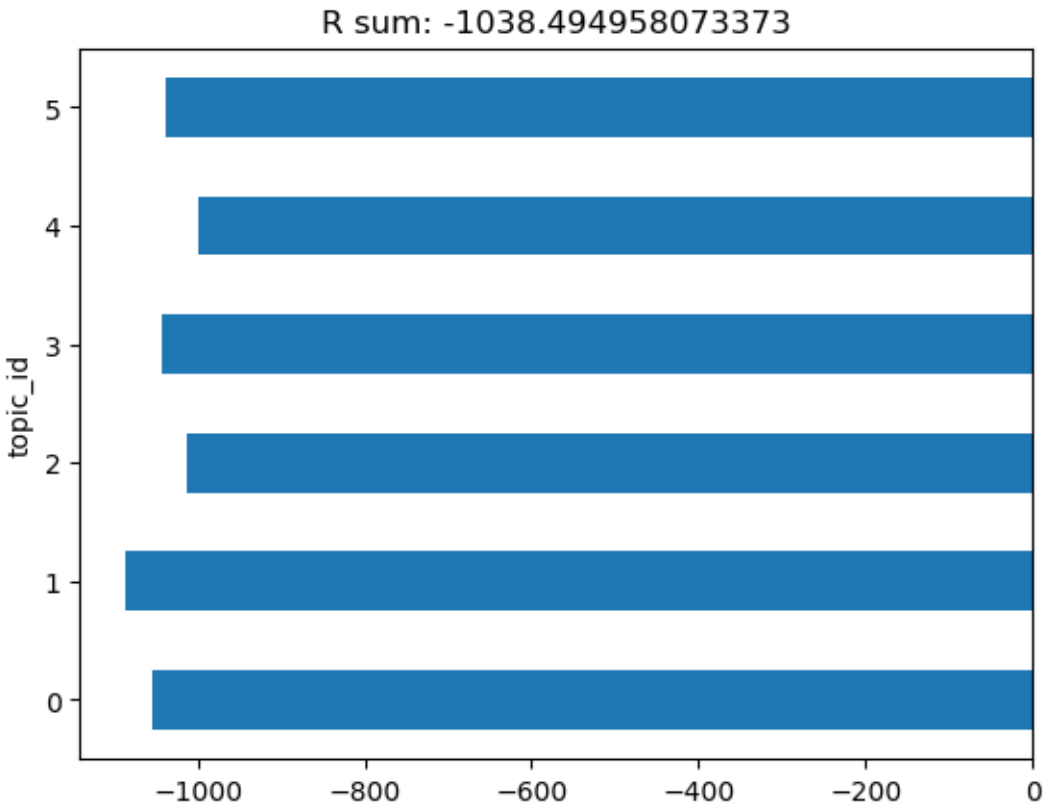
```
compute_entropy(THETA.T)
```



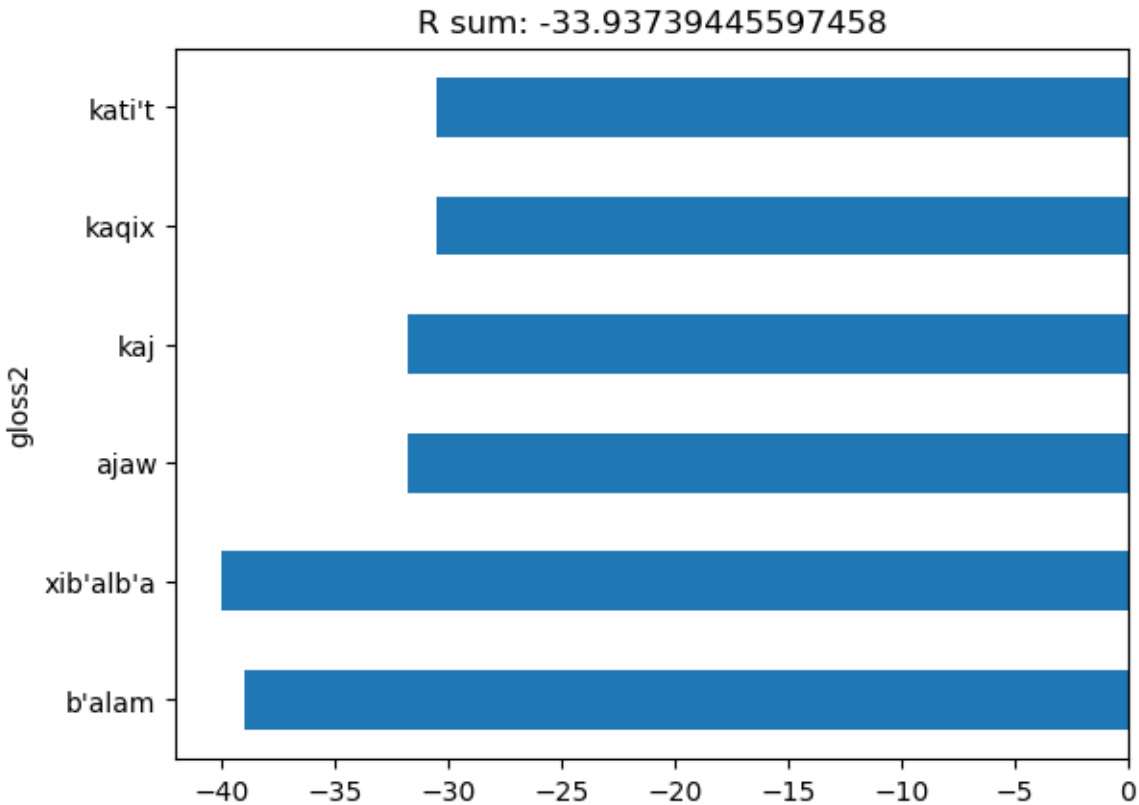

```
# R.mean()
```

```
60 | 0.4262047671312379
30 | 0.34843081567879547
```

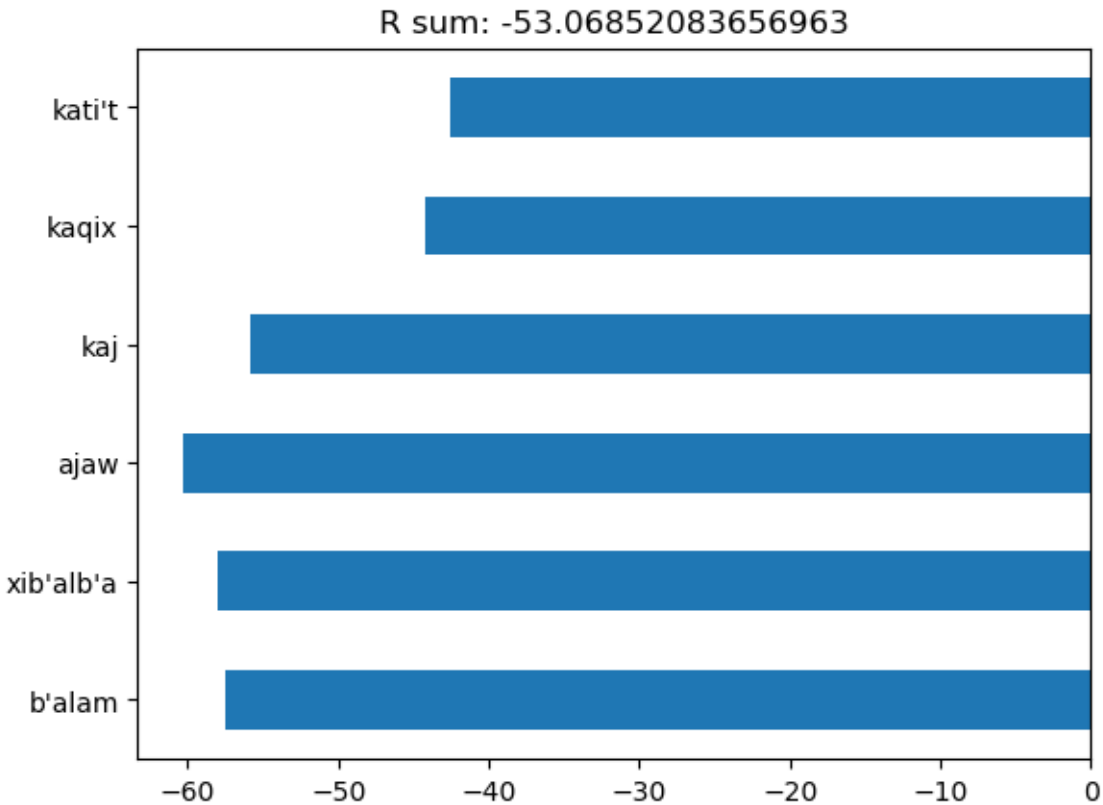
```
compute_entropy(PHI.T)
```



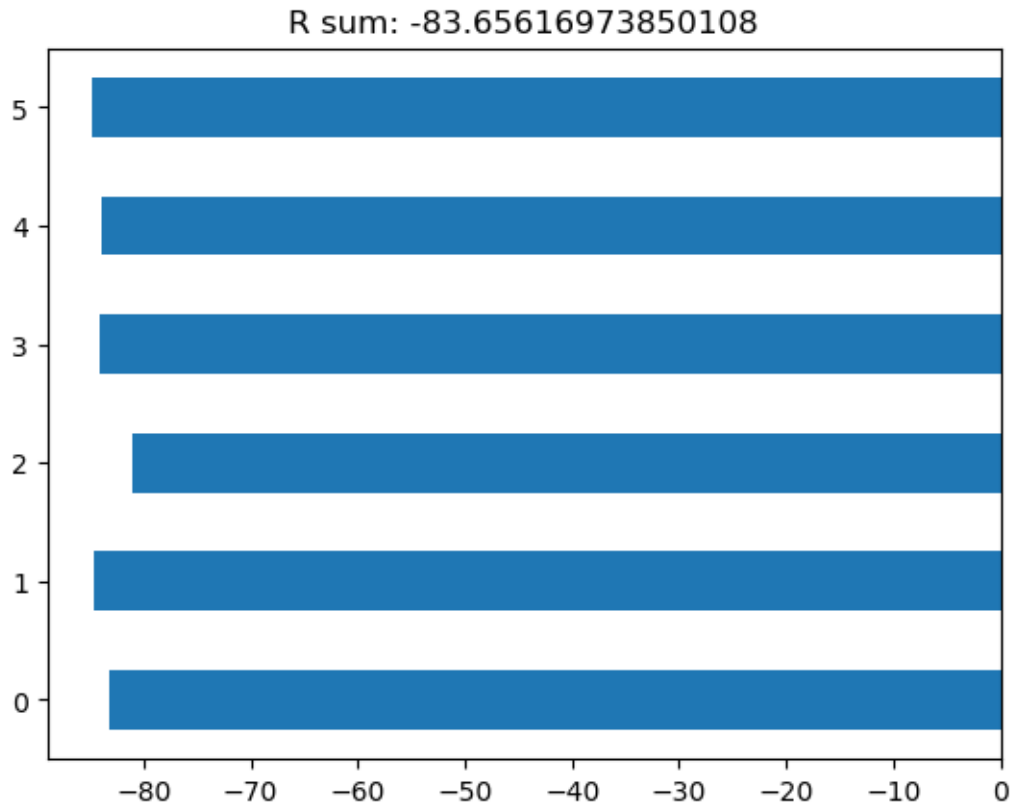

```
compute_entropy(THETA)
```




```
compute_entropy(CHAP_TOPIC_THETA)
```




```
compute_entropy(CHAP_TOPIC)
```


R sum: -83.65616973850108
5
4
3
2
1
0
−80
−70
−60
−50
−40
−30
−20
−10
0